

LAPORAN PRAKTIKUM

STRUKTUR DATA

Pertemuan 3



Disusun oleh :

Nama : Salwachika Deri
Nim : 25071102204
Dosen : Reny Fitri Yani, S.T., M.T
Asdos : -Akhlaqul Muhammad Fadwa (2307112834)
-Rizkillah Ramanda Sinyo (2307126144)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS RIAU

GENAP

PEKANBARU

2026

A. Python Object-Oriented Programming

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang fokus pada **objek**. Objek = representasi sesuatu di dunia nyata yang mempunyai **data (atribut)** dan **perilaku (metode)**. Python adalah bahasa berorientasi objek, yang memungkinkan Anda untuk menyusun kode menggunakan kelas dan objek untuk pengorganisasian dan penggunaan kembali yang lebih baik. Keunggulan OOP :

- Memberikan struktur yang jelas pada program.
- Membuat kode lebih mudah dipelihara, digunakan kembali, dan di-debug.
- Membantu menjaga agar kode Anda tetap DRY (**Jangan Ulangi Diri Sendiri**)
- Memungkinkan Anda membangun aplikasi yang dapat digunakan kembali dengan lebih sedikit kode.

B. Python Classes/Objects

Kelas dan objek adalah dua konsep inti dalam pemrograman berorientasi objek. Sebuah kelas mendefinisikan seperti apa seharusnya sebuah objek, dan sebuah objek dibuat berdasarkan kelas tersebut. Misalnya:

Class	Objects
Fruit	Apple, Banana, Mango
Car	Apple, Banana, Mango

Saat Anda membuat objek dari suatu kelas, objek tersebut akan mewarisi semua variable didefinisikan di dalam kelas tersebut.

- Untuk membuat kelas, gunakan kata kunci **class**:
contoh :

```
class MyClass:  
    x = 5
```

- Untuk membuat objek, kita dapat menggunakan kelas bernama MyClass :

```
Buat objek bernama p1, dan cetak nilai x :  
p1 = MyClass()  
print(p1.x)
```

- Untuk menghapus objek, kita dapat menggunakan kata kunci del :

```
del p1
```

- Dapat membuat beberapa objek dari kelas yang sama, Setiap objek bersifat independen dan memiliki salinan properti kelasnya sendiri. contoh :

Buat tiga objek dari kelas MyClass:

```
p1 = MyClass()  
p2 = MyClass()  
p3 = MyClass()
```

```
print(p1.x)  
print(p2.x)  
print(p3.x)
```

- Pass Statement : class Definisi tidak boleh kosong, tetapi jika karena suatu alasan Anda memiliki class definisi tanpa konten, tambahkan pass pernyataan tersebut untuk menghindari kesalahan. contoh :

```
class Person:  
    pass
```

C. Python `__init__()` Method

Semua kelas memiliki metode bawaan yang disebut `__init__()`, yang selalu dieksekusi ketika kelas tersebut diinisialisasi. Metode ini `__init__()` digunakan untuk menetapkan nilai pada properti objek, atau untuk melakukan operasi yang diperlukan saat objek sedang dibuat. Metode ini `__init__()` dipanggil secara otomatis setiap kali kelas digunakan untuk membuat objek baru. Contoh :

Buat kelas bernama Person, gunakan `__init__()` metode tersebut untuk menetapkan nilai untuk nama dan usia:

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
p1 = Person("Emil", 36)
```

```
print(p1.name)  
print(p1.age)
```

Tanpa menggunakan `__init__()` Method, kita harus mengatur property secara manual untuk setiap objek seperti pada contoh berikut :

```
class Person:
    pass

p1 = Person()
p1.name = "Tobias"
p1.age = 25

print(p1.name)
print(p1.age)
```

penggunaan `__init__()` method dapat mempermudah pembuatan objek dengan nilai awal. perhatikan contoh berikut :

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 28)

print(p1.name)
print(p1.age)
```

kita dapat mengatur nilai default untuk parameter dalam `__init__()` method. contoh:

Tetapkan nilai default untuk parameter usia:

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

p1 = Person("Emil")
p2 = Person("Tobias", 25)

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Metode `__init__()` dapat memiliki parameter sebanyak yang dibutuhkan:

```
class Person:
    def __init__(self, name, age, city, country):
        self.name = name
        self.age = age
        self.city = city
        self.country = country
```

```
p1 = Person("Linus", 30, "Oslo", "Norway")

print(p1.name)
print(p1.age)
print(p1.city)
print(p1.country)
```

D. Python Self Parameter

Parameter self merupakan referensi ke instance kelas saat ini. Ini digunakan untuk mengakses properti dan metode yang dimiliki oleh kelas tersebut. Parameter self tersebut harus merupakan parameter pertama dari metode mana pun dalam kelas tersebut. Digunakan self untuk mengakses properti kelas. Tanpa itu self, Python tidak akan tahu properti objek mana yang ingin Anda akses: contoh:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print("Hello, my name is " + self.name)

p1 = Person("Emil", 25)
p1.greet()
```

Tidak harus diberi nama self, kita bisa menyebutnya apa pun, tetapi harus menjadi parameter pertama dari metode apa pun di dalam kelas tersebut. contoh :

```
class Person:
    def __init__(myobject, name, age):
        myobject.name = name
        myobject.age = age

    def greet(abc):
        print("Hello, my name is " + abc.name)

p1 = Person("Emil", 36)
p1.greet()
```

catatan : Meskipun dapat menggunakan nama yang berbeda, sangat disarankan untuk menggunakan nama tersebut self, karena merupakan konvensi dalam Python dan membuat kode Anda lebih mudah dibaca oleh orang lain

E. Python Class Properties

Properti adalah variabel yang dimiliki oleh suatu kelas. Variabel ini menyimpan data untuk setiap objek yang dibuat dari kelas tersebut.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Emil", 36)

print(p1.name)
print(p1.age)
```

-Akses Properti menggunakan notasi titik:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

car1 = Car("Toyota", "Corolla")

print(car1.brand)
print(car1.model)
```

-Ubah property dapat dengan memodifikasi nilai properti pada objek:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Tobias", 25)
print(p1.age)

p1.age = 26
print(p1.age)
```

-hapus properti menggunakan kata kunci del:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 30)
```

```
del p1.age  
  
print(p1.name)
```

F. Python Class Methods

Metode adalah fungsi yang dimiliki oleh suatu kelas. Metode mendefinisikan perilaku objek yang dibuat dari kelas tersebut. Semua metode harus memiliki self parameter pertama.

-metode dapat menerima parameter :

```
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
    def multiply(self, a, b):  
        return a * b  
  
calc = Calculator()  
print(calc.add(5, 3))  
print(calc.multiply(4, 7))
```

-metode yang mengakses property objek :

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def get_info(self):  
        return f"{self.name} is {self.age} years old"  
  
p1 = Person("Tobias", 28)  
print(p1.get_info())
```

